

REAL ESTATE BUILDER ERP BACKEND

Complete Technical Documentation

Database Schema • API Reference • Integration Guide

Multi-Tenant SaaS Platform for Real Estate Developers

Pakistan • India • South Asia

66	37	8	7,600+
API Endpoints	Database Models	User Roles	Lines of Code

github.com/faizanakram2/real-estate-erp

Table of Contents

- 1. Overview
- 2. Technology Stack
- 3. Project Setup
- 4. Database Schema
- 5. Authentication & Authorization
- 6. API Reference
 - 6.1 Dashboard & Search
 - 6.2 Projects, Blocks & Plots
 - 6.3 Customers & CRM
 - 6.4 Bookings & Sales
 - 6.5 Installments & Payments
 - 6.6 Construction Tracking
 - 6.7 HR & Payroll
 - 6.8 Vendors & Materials
 - 6.9 Maintenance
 - 6.10 Documents & Files
 - 6.11 Accounting (Ledger, Transactions)
 - 6.12 Commissions
 - 6.13 Notifications & Reminders
 - 6.14 Reports & Analytics
 - 6.15 Bulk Operations
 - 6.16 Customer Portal
 - 6.17 Organization Settings
 - 6.18 Audit & Activity
- 7. User Roles Matrix
- 8. Error Handling
- 9. Frontend Integration Guide
- 10. Deployment Notes

1. Overview

Real Estate Builder ERP is a comprehensive, multi-tenant SaaS platform built specifically for real estate developers, builders, and housing societies in Pakistan, India, and the broader South Asian market.

The system addresses the unique challenges of the real estate construction industry, including plot inventory management, flexible installment plans, manual payment recording with verification workflows, construction progress tracking, customer portals, and full accounting — all within a unified, role-based platform.

Key Modules

Module	Description
Land & Plot Management	Projects, blocks, plots with map data, dimensions, pricing
Customer/CRM	Complete customer lifecycle with portal access
Booking & Sales	Auto-generated booking numbers and installment schedules
Installment Plans	1Y, 3Y, 5Y flexible plans with monthly/quarterly frequency
Payment Management	Multi-method, proof uploads, verification workflow
Construction Tracking	Phases, updates, material requisitions
HR & Payroll	Employees, attendance, monthly payroll generation
Accounting	Chart of accounts, transactions, P&L, cash flow
Commissions	Sales agent commission tracking and payouts
Customer Portal	Self-service: bookings, installments, receipts
Notifications	SMS, WhatsApp, Email templates with auto-reminders
Reports & Analytics	Collection forecast, aging, recovery, ledger

2. Technology Stack

Component	Technology
Framework	Next.js 14 (App Router, Server Actions)
Language	TypeScript 5.x
Database	PostgreSQL 14+
ORM	Prisma 5.22
Authentication	NextAuth.js (JWT strategy) + custom JWT for customer portal
Validation	Zod schemas
Password Hashing	bcryptjs (12 rounds)
File Upload	Local filesystem (S3-ready architecture)
Styling (optional)	Tailwind CSS 3.x (UI layer owned by frontend team)
Deployment Target	Vercel, AWS, DigitalOcean, or any Node.js-capable host

Architecture Principles

Multi-Tenancy: Every data model is scoped to an Organization. Users cannot access data across organizations.

Role-Based Access Control (RBAC): Centralized permission map in `src/lib/rbac.ts` with 8 roles.

Audit Trail: All mutating operations can be logged to `AuditLog` via the `logAudit()` helper.

Type Safety: Prisma-generated types + Zod input validation for end-to-end type safety.

3. Project Setup

Requirements

- Node.js 22.12+ or 20.19+
- PostgreSQL 14 or higher
- npm (or yarn/pnpm)

Installation Steps

```
# 1. Clone the repository
git clone https://github.com/faizanakram2/real-estate-erp.git
cd real-estate-erp

# 2. Install dependencies
npm install

# 3. Setup environment
cp .env.example .env
# Edit .env and set DATABASE_URL to your PostgreSQL connection string

# 4. Push database schema
npm run db:push

# 5. Generate Prisma client
npm run db:generate

# 6. Seed sample data (organizations, users, plots, customers, bookings)
npm run db:seed

# 7. Start dev server
npm run dev
# Server starts on http://localhost:3000
```

Environment Variables

```
# Database (Required)
DATABASE_URL="postgresql://user:password@localhost:5432/real_estate_erp"

# NextAuth (Required)
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="<generate with: openssl rand -base64 32>"

# App
NEXT_PUBLIC_APP_URL="http://localhost:3000"
NEXT_PUBLIC_APP_NAME="Real Estate Builder ERP"

# File Storage (Optional)
UPLOAD_DIR="./uploads"
MAX_FILE_SIZE_MB=10
```

Demo Login Credentials (after seeding)

Role	Email	Password	Access
Admin	admin@devlayers.org	admin123	Full system access

Manager	manager@devlayers.org	manager123	Projects, bookings, approvals
Sales Agent	sales@devlayers.org	sales123	Create bookings, manage customers
Accountant	accounts@devlayers.org	accounts123	Payments, ledger, payroll
Site Engineer	engineer@devlayers.org	engineer123	Construction, materials

4. Database Schema

The database uses PostgreSQL with Prisma ORM. Total: **37 models** organized into logical modules.

Multi-Tenancy & Authentication

Model	Purpose
Organization	Top-level tenant (builder/developer company)
OrganizationBankAccount	Bank details shown to customers for payments
User	Staff members with roles (admin, manager, sales, etc.)
Account	OAuth accounts (NextAuth)
Session	User sessions (NextAuth)
VerificationToken	Email verification tokens

Property Management

Model	Purpose
Project	Housing society or commercial project
Block	Sub-division within a project (Block A, B, C)
Plot	Individual plot with pricing, size, status
PlotImage	Photos associated with a plot
PlotPriceHistory	Track price changes with reason and audit

CRM & Sales

Model	Purpose
Customer	Buyers with CNIC, portal access, emergency contact, nominee
Booking	Plot booking with installment schedule
InstallmentPlan	Reusable plans (1Y, 3Y, 5Y) with penalty config
Installment	Individual installment (pending, paid, overdue)
PaymentRecord	Payment with verification workflow
Commission	Sales agent commission tracking

Construction & Operations

Model	Purpose
ConstructionPhase	Foundation, Structure, Finishing, etc.
ConstructionUpdate	Progress update with photos
ConstructionImage	Photos attached to updates
MaterialRequisition	Material orders with vendors

MaintenanceRequest	Work orders with priority, assignee
MaintenanceImage	Before/during/after photos
MaintenanceComment	Internal/external comments thread
Vendor	Plumbers, electricians, suppliers

HR & Payroll

Model	Purpose
Employee	Staff with salary, department, employment type
Attendance	Daily attendance with GPS and hours worked
Payroll	Monthly payroll with allowances, deductions, tax

Accounting

Model	Purpose
LedgerAccount	Chart of accounts (ASSET, LIABILITY, INCOME, EXPENSE)
Transaction	Income/expense/transfer entries
TransactionAttachment	Receipts and proofs

Documents, Notifications & System

Model	Purpose
Document	Files linked to customer/booking/project/vendor
NotificationTemplate	Reusable SMS/WhatsApp/Email templates
Notification	In-app notifications per user
Note	Notes attached to customers or properties
AuditLog	All mutating actions logged for compliance
SystemSetting	Key-value configuration store

Key Enums

Enum	Values
UserRole	SUPER_ADMIN, ADMIN, MANAGER, SALES_AGENT, ACCOUNTANT, SITE_ENGINEER, STAFF, CUSTOMER
PlotStatus	AVAILABLE, BOOKED, SOLD, TRANSFERRED, CANCELLED, ON_HOLD, RESERVED
PlotType	RESIDENTIAL, COMMERCIAL, INDUSTRIAL, FARMHOUSE, APARTMENT, SHOP, OFFICE, PENTHOUSE
BookingStatus	BOOKED, CONFIRMED, ACTIVE, POSSESSION_GIVEN, TRANSFERRED, CANCELLED, DEFAULTER
InstallmentStatus	PENDING, PARTIAL, PAID, OVERDUE, WAIVED
PaymentMethod	CASH, BANK_TRANSFER, CHEQUE, JAZZCASH, EASYPAYSA, ONLINE, DEMAND_DRAFT, PAY_ORDER
PaymentStatus	PENDING, VERIFIED, CONFIRMED, REJECTED, REFUNDED, CANCELLED

5. Authentication & Authorization

Two Authentication Systems

1. **Staff Portal (NextAuth):** JWT-based, cookie sessions, for admin/manager/sales/etc.
2. **Customer Portal:** Custom JWT (Bearer token), for end customers only.

Staff Login Flow (NextAuth)

```
POST /api/auth/callback/credentials
Content-Type: application/json

{
  "email": "admin@devlayers.org",
  "password": "admin123"
}

// Frontend code:
import { signIn } from "next-auth/react";

const result = await signIn("credentials", {
  email: "admin@devlayers.org",
  password: "admin123",
  redirect: false,
});

// Session available via:
import { useSession } from "next-auth/react";
const { data: session } = useSession();
// session.user = { id, email, name, role, organizationId, organizationName }
```

Customer Portal Login Flow

```
// Step 1: Register (if first time)
POST /api/customer-portal/auth
{
  "action": "register",
  "phone": "+92-321-1234567",
  "cnic": "35201-1234567-1",
  "password": "yourpassword"
}

// Step 2: Login
POST /api/customer-portal/auth
{
  "phone": "+92-321-1234567",
  "password": "yourpassword"
}

// Response:
{
  "token": "eyJhbGciOi...",
  "customer": { id, firstName, lastName, phone, email, organization }
}

// Use token in subsequent requests:
GET /api/customer-portal/my-bookings
```

```
Authorization: Bearer <token>
```

RBAC Permissions

Permissions are centralized in `src/lib/rbac.ts`. Use `authorize(permission)` in routes:

```
// Route example
import { authorize, isErrorResponse } from "@lib/rbac";

export async function POST(request: NextRequest) {
  const auth = await authorize("bookings:write");
  if (isErrorResponse(auth)) return auth; // returns 401/403
  const { session } = auth; // contains user + org
  // ... your logic
}
```

6. User Roles Matrix

Role	Permissions
SUPER_ADMIN	Full system access, cross-organization (platform owner)
ADMIN	Organization admin: all modules, user management, settings, audit logs
MANAGER	Projects, bookings, customers, reports, approvals, user listing
SALES_AGENT	Customers CRUD, bookings create/read, plot listing, payments create/read
ACCOUNTANT	Payments verify/confirm, ledger, transactions, payroll, reports
SITE_ENGINEER	Construction updates, material requisitions, attendance
STAFF	Basic read access to projects, plots, documents
CUSTOMER	Customer portal only (own bookings, installments, payments)

7. API Reference

Base URL: `http://localhost:3000/api`

All endpoints require authentication unless noted. Staff endpoints use NextAuth cookies, customer portal endpoints use Bearer tokens.

Common Response Formats

Paginated list response:

```
{
  "data": [...],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8,
    "hasNext": true,
    "hasPrev": false
  }
}
```

Error response:

```
{
  "error": "Validation failed",
  "details": { "fieldErrors": {...}, "formErrors": [...] }
}
```

Common query parameters for list endpoints:

Param	Type	Description
page	number	Page number (default: 1)
limit	number	Items per page (default: 20, max: 100)
search	string	Search term (endpoint-specific fields)
sortBy	string	Sort field (default: createdAt)
sortOrder	asc/desc	Sort direction (default: desc)

Complete Endpoint Reference

Dashboard & Search

GET	/api/dashboard	Stats, recent payments, overdue installments
GET	/api/search?q=	Global search (customers, plots, bookings, payments, projects)

Projects & Blocks

GET	/api/projects	List projects (filter: status, type)
POST	/api/projects	Create project
GET	/api/projects/:id	Project with blocks, plots, construction phases
PATCH	/api/projects/:id	Update project
DELETE	/api/projects/:id	Delete project (must have no plots)
GET	/api/projects/:id/blocks	List blocks with plot stats
POST	/api/projects/:id/blocks	Create block

Plots

GET	/api/plots	List (filter: projectId, blockId, status, type)
POST	/api/plots	Create single OR bulk (bulk: true)
GET	/api/plots/:id	Plot with bookings, images, maintenance
PATCH	/api/plots/:id	Update plot

Customers

GET	/api/customers	List customers (filter: status)
POST	/api/customers	Create customer
GET	/api/customers/:id	Customer with bookings, payments, documents
PATCH	/api/customers/:id	Update customer
GET	/api/customers/check-duplicate	Check by phone/cnic/email before create

Bookings & Sales

GET	/api/bookings List bookings (filter: status, projectId, customerId)
POST	/api/bookings Create booking (auto-generates installments)
GET	/api/bookings/:id Booking with installments, payments, summary
PUT	/api/bookings/:id Transfer to new customer
DELETE	/api/bookings/:id Cancel booking (releases plot)
GET	/api/bookings/:id/early-settlement Calculate settlement amount
POST	/api/bookings/:id/early-settlement Execute early settlement
POST	/api/bookings/:id/defaulters Manually flag as defaulter
PUT	/api/bookings/:id/defaulters Auto-detect defaulters (3+ overdue)

Installment Plans

GET	/api/installment-plans List active plans
POST	/api/installment-plans Create plan (duration, downpayment %, penalty %)

Installments

GET	/api/installments/overdue Overdue list with aging buckets
POST	/api/installments/overdue Mark past-due as overdue + calc penalty
PATCH	/api/installments/:id/reschedule Move due date
PATCH	/api/installments/:id/waive-penalty Waive penalty (full/partial)
POST	/api/installments/:id/split Split 1 installment into 2-12 parts

Payments

GET	/api/payments List (filter: status, bookingId, customerId)
POST	/api/payments Record payment (auto receipt number)
GET	/api/payments/:id Payment details
PATCH	/api/payments/:id Verify / Confirm / Reject

Construction

GET	/api/projects/:id/construction-phases List phases
POST	/api/projects/:id/construction-phases Create phase
GET	/api/construction-phases/:id Phase with updates, materials
PATCH	/api/construction-phases/:id Update progress/status
POST	/api/construction-phases/:id/updates Post update with images
GET	/api/construction-phases/:id/material-requisitions List requisitions
POST	/api/construction-phases/:id/material-requisitions Create requisition

HR & Payroll

GET	/api/employees List employees (filter: department, status)
POST	/api/employees Create employee
GET	/api/employees/:id Employee with attendance, payroll
PATCH	/api/employees/:id Update employee
GET	/api/employees/:id/attendance Attendance (filter: month, year)
POST	/api/employees/:id/attendance Record attendance with GPS
GET	/api/employees/:id/payroll Payroll history
POST	/api/employees/:id/payroll Generate monthly payroll

Vendors

GET	/api/vendors List (filter: category)
POST	/api/vendors Create vendor
GET	/api/vendors/:id Vendor with maintenance, transactions
PATCH	/api/vendors/:id Update vendor
DELETE	/api/vendors/:id Deactivate (soft delete)

Maintenance

GET	/api/maintenance-requests List (filter: status, priority)
------------	---

POST	/api/maintenance-requests	Create work order
GET	/api/maintenance-requests/:id	Request with images, comments
PATCH	/api/maintenance-requests/:id	Assign, change status
POST	/api/maintenance-requests/:id/comments	Add comment

Documents & Files

GET	/api/documents	List (filter: type, projectId, customerId)
POST	/api/documents	Create document record
GET	/api/documents/:id	Document details
DELETE	/api/documents/:id	Delete document
POST	/api/upload	Upload file (multipart, 10MB max)

Accounting

GET	/api/ledger-accounts	List chart of accounts
POST	/api/ledger-accounts	Create account
GET	/api/transactions	List transactions (filter: type, date range)
POST	/api/transactions	Create transaction

Commissions

GET	/api/commissions	List with summary (pending/paid amounts)
POST	/api/commissions	Create commission (% or fixed)
PATCH	/api/commissions/:id	Approve / Pay / Cancel

Notifications & Reminders

GET	/api/notifications	User notifications (filter: unreadOnly)
PATCH	/api/notifications	Mark all as read
PATCH	/api/notifications/:id	Mark single as read
DELETE	/api/notifications/:id	Delete notification

GET /api/notification-templates
List templates

POST /api/notification-templates
Create template

GET /api/reminders
Preview pending reminders (count)

POST /api/reminders
Generate all payment reminders

Reports & Analytics

GET /api/reports/collection-forecast
Expected vs actual by month

GET /api/reports/overdue-aging
30/60/90+ aging report

GET /api/reports/recovery-rate
By project and sales agent

GET /api/reports/customer-ledger/:id
Complete payment history

GET /api/reports/plot-inventory
Inventory by status, type, project

GET /api/reports/profit-loss
P&L with income/expenses breakdown

GET /api/reports/cash-flow
Monthly cash flow (inflows vs outflows)

Bulk Operations

POST /api/bulk/attendance
Mark 50+ employees at once

POST /api/bulk/plot-prices
Update prices by % or fixed (with history)

Data Export

GET /api/export/customers
CSV export of customers

GET /api/export/plots
CSV export of plots

GET /api/export/bookings
CSV export of bookings

GET /api/export/payments
CSV export of payments

GET /api/export/installments
CSV export of installments

GET /api/export/employees
CSV export of employees

Organization Settings

GET /api/organization
Get org profile with bank accounts

PATCH	/api/organization Update org profile
GET	/api/organization/bank-accounts List active bank accounts
POST	/api/organization/bank-accounts Add bank account
PATCH	/api/organization/bank-accounts/:id Update bank account
DELETE	/api/organization/bank-accounts/:id Deactivate bank account

Users

GET	/api/users List users (Admin/Manager only)
POST	/api/users Create user (Admin only)

Audit & Activity

GET	/api/audit-logs View audit logs (Admin only)
GET	/api/activity/:type/:id Timeline for any entity

Customer Portal

POST	/api/customer-portal/auth Register (phone+CNIC) / Login (phone+pwd)
GET	/api/customer-portal/my-bookings Customer's own bookings
GET	/api/customer-portal/my-installments Schedule with next-due
GET	/api/customer-portal/my-payments Payment history

8. Sample Request/Response Examples

Create Project

```
POST /api/projects
Content-Type: application/json

{
  "name": "Green Valley Housing Society",
  "type": "HOUSING_SOCIETY",
  "city": "Lahore",
  "state": "Punjab",
  "totalArea": 500,
  "areaUnit": "kanal",
  "totalPlots": 200,
  "startDate": "2026-01-15T00:00:00.000Z",
  "totalBudget": 500000000
}
```

Bulk Create Plots

```
POST /api/plots

{
  "bulk": true,
  "projectId": "clxxx...",
  "blockId": "clyyy...",
  "plotPrefix": "A",
  "startNumber": 1,
  "count": 40,
  "type": "RESIDENTIAL",
  "size": 5,
  "sizeUnit": "marla",
  "basePrice": 2500000
}
```

Create Booking with Installment Schedule

```
POST /api/bookings

{
  "customerId": "clxxx...",
  "plotId": "clyyy...",
  "totalPrice": 3000000,
  "bookingAmount": 150000,
  "downPayment": 600000,
  "installmentPlanId": "clzzz...",
  "installmentMonths": 36
}

// Auto-generates: booking number (BK-2026-0001),
// installment schedule (38 entries: booking + down + 36 monthly)
```

Record & Verify Payment

```
// Step 1: Record payment (any role with payments:write)
POST /api/payments

{
```

```

"customerId": "clxxx...",
"bookingId": "clyyy...",
"installmentId": "clzzz...",
"amount": 100000,
"paymentMethod": "BANK_TRANSFER",
"paymentDate": "2026-04-10T00:00:00.000Z",
"referenceNumber": "TXN-123456",
"bankName": "HBL"
}
// Response: PaymentRecord with status "PENDING"

// Step 2: Verify (Accountant/Manager/Admin)
PATCH /api/payments/:id
{
  "status": "CONFIRMED" // or "REJECTED" with rejectionReason
}
// On CONFIRMED: installment.paidAmount updates, status -> PAID/PARTIAL

```

File Upload

```

// Frontend
const formData = new FormData();
formData.append("file", selectedFile);

const res = await fetch("/api/upload", {
  method: "POST",
  body: formData,
});

const { fileUrl, fileName, fileSize, mimeType } = await res.json();

// Then link it to a document record:
await fetch("/api/documents", {
  method: "POST",
  body: JSON.stringify({
    name: fileName,
    type: "PAYMENT_RECEIPT",
    fileUrl, fileSize, mimeType,
    bookingId: "clxxx..."
  })
});

```

9. Error Handling

Code	Meaning	When
200	OK	Successful GET/PATCH
201	Created	Successful POST (resource created)
400	Bad Request	Validation error or invalid input
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but role lacks permission
404	Not Found	Resource does not exist or not in user's org
500	Server Error	Unexpected error (check server logs)

Validation errors from Zod include field-level details:

```
{
  "error": "Validation failed",
  "details": {
    "fieldErrors": {
      "phone": ["Phone must be at least 10 characters"],
      "email": ["Invalid email address"]
    },
    "formErrors": []
  }
}
```

10. Frontend Integration Guide

Setup

Wrap your app with NextAuth provider:

```
// app/providers.tsx
"use client";
import { SessionProvider } from "next-auth/react";

export function Providers({ children }) {
  return <SessionProvider>{children}</SessionProvider>;
}

// app/layout.tsx
import { Providers } from "../providers";
export default function RootLayout({ children }) {
  return (
    <html><body>
      <Providers>{children}</Providers>
    </body></html>
  );
}
```

Making Authenticated Requests

Session cookies are automatic. Use fetch or axios directly:

```
// Simple fetch - cookie auth handled automatically
const res = await fetch("/api/projects");
const { data, pagination } = await res.json();

// POST with JSON body
const res = await fetch("/api/customers", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(customerData),
});
if (!res.ok) {
  const { error, details } = await res.json();
  // handle error
}
```

Conditional UI by Role

```
import { useSession } from "next-auth/react";

function BookingActions() {
  const { data: session } = useSession();
  const role = session?.user?.role;

  return (
    <>
      {role === "MANAGER" || role === "ADMIN" ? (
        <button>Cancel Booking</button>
      ) : null}
      {["ACCOUNTANT", "MANAGER", "ADMIN"].includes(role) && (
        <button>Verify Payment</button>
      )}
    </>
  );
}
```

```
  })  
</>  
  );  
}
```

Customer Portal Integration

```
// Store JWT in localStorage after login  
localStorage.setItem("customer_token", token);  
  
// Attach to all customer portal requests:  
const res = await fetch("/api/customer-portal/my-bookings", {  
  headers: {  
    "Authorization": `Bearer ${localStorage.getItem("customer_token")}`  
  }  
});
```

11. Deployment Notes

Production Checklist

- Set NEXTAUTH_SECRET to a cryptographically secure value (openssl rand -base64 32)
- Set NEXTAUTH_URL to your production domain (https://yourdomain.com)
- Configure DATABASE_URL with connection pooling (PgBouncer recommended)
- Use HTTPS-only cookies in production (NextAuth does this automatically)
- Replace local /uploads storage with S3/CloudFront for production
- Set up automated DB backups (daily minimum)
- Configure rate limiting at proxy/CDN level (Cloudflare, AWS WAF)
- Setup Sentry or similar for error tracking
- Run 'npm run db:migrate deploy' for production migrations (not db:push)
- Setup cron job for POST /api/installments/overdue (daily at midnight)
- Setup cron job for POST /api/reminders (daily at 9am)
- Integrate SMS gateway (Twilio, Jazz SMS) to send actual messages
- Integrate WhatsApp Business API for notifications

Recommended Cron Schedule

```
# Daily at 00:00 - mark overdue installments
0 0 * * * curl -X POST https://yourdomain.com/api/installments/overdue

# Daily at 09:00 - generate reminders
0 9 * * * curl -X POST https://yourdomain.com/api/reminders

# Weekly on Sunday - detect defaulters across all bookings
0 0 * * 0 curl -X PUT https://yourdomain.com/api/bookings/all/defaulters
```

12. Project File Structure

```
real-estate-erp/
|-- prisma/
| |-- schema.prisma # 37 models, 23 enums
| |-- seed.ts # Sample data: org, users, plots, bookings
|
|-- src/
| |-- app/
| | |-- api/ # 66 API route handlers
| | | |-- auth/ # NextAuth routes
| | | |-- customer-portal/ # Customer JWT endpoints
| | | |-- projects/ # Projects CRUD
| | | |-- plots/ # Plots CRUD + bulk
| | | |-- customers/ # Customer CRUD + dupe check
| | | |-- bookings/ # Bookings + settlement + defaulter
| | | |-- installments/ # Reschedule, split, waive, overdue
| | | |-- payments/ # Payments + verification
| | | |-- commissions/ # Sales commission tracking
| | | |-- reports/ # 7 report endpoints
| | | |-- export/ # CSV exports
| | | |-- bulk/ # Bulk operations
| | | |-- activity/ # Entity timeline
| | | |-- search/ # Global search
| | | |-- reminders/ # Payment reminder generator
| | | |-- ... # (and more)
| | |-- layout.tsx
| | |-- page.tsx
| |
| |-- lib/
| | |-- prisma.ts # Prisma client singleton
| | |-- auth.ts # NextAuth config
| | |-- api-utils.ts # Auth helpers, pagination
| | |-- rbac.ts # Permission map (8 roles)
| | |-- audit.ts # Audit logging helper
| | |-- customer-auth.ts # Customer JWT verification
| | |-- utils.ts # Currency, date formatters
| | |-- validators/ # Zod schemas per entity
| |
| |-- types/
| |-- index.ts
|
|-- API_DOCUMENTATION.md # Markdown API reference
|-- BACKEND_DOCUMENTATION.pdf # This file
|-- .env.example
|-- package.json # Scripts: dev, build, db:*
|-- README.md
```

Summary

This backend provides a production-ready foundation for a Real Estate Builder ERP tailored to the Pakistani and South Asian market. All 66 endpoints are compiled, authorized via role-based permissions, and backed by a comprehensive 37-model database schema designed around the realities of plot-based real estate development, installment-driven sales, and multi-stakeholder operations.

Build & Verification Status

Component	Status
Prisma Schema	37 models, 23 enums, fully indexed
Prisma Client	Generated successfully
Next.js Build	Compiled successfully (66 dynamic routes)
Dev Server	Starts on port 3000 with HTTP 200
Authentication	NextAuth providers endpoint verified
RBAC	Protected routes return 401 without auth (verified)
Type Safety	TypeScript strict mode, Zod validation
Git Repository	Pushed to github.com/faizanakram2/real-estate-erp

Repository: github.com/faizanakram2/real-estate-erp

End of Documentation